Hierarchical Softmax and Huffman Coding

Solving the $O(|V|)$ Bottleneck in Word Embeddings

James Pustejovsky

Brandeis University

February 3, 2026

The Computational Bottleneck

- Standard Softmax requires a summation over the entire vocabulary $|V|$.
- For $|V| = 10^5$, calculating $P(c|w)$ for every training step is $O(|V|)$.
- **Goal:** Reduce complexity to $O(\log |V|)$ using a binary tree structure.

Binary Tree Representation

- Words ($w \in V$) are represented as **leaf nodes**.
- There is a unique path from the root to each word w.
- Let $L(w)$ be the length of the path to word w.
- We replace the flat output layer with a series of binary classification steps.

The Probability Calculation

The probability of word w is the product of probabilities of binary turns:

$$P(w|w_I) = \prod_{j=1}^{L(w)-1} \sigma([n(w,j+1) == \text{ch}(n(w,j))] \cdot \theta_{n(w,j)}^T v_{w_I})$$

- $n(w,j)$ is the j-th node on the path to w.
- $\theta_{n(w,j)}$ is a learned vector for the internal node.
- σ is the sigmoid function, deciding "left" vs. "right".

Optimization via Huffman Trees

Why Huffman Trees?

A balanced binary tree gives $O(\log_2 |V|)$. A **Huffman Tree** optimizes for word frequency.

- Frequent words (e.g., "the", "of") are assigned **shorter paths**.
- Rare words (e.g., "quaternion") are assigned **longer paths**.
- This minimizes the *expected* number of calculations per corpus token.

Huffman Tree Construction

1. Calculate frequencies for all words in the corpus.
2. Treat each word as a leaf node.
3. Repeatedly merge the two nodes with the lowest frequencies into a new internal node.
4. The final root encompasses all words; path lengths inversely correlate with frequency.

Summary: Hierarchical vs. Flat Softmax

Feature	**Flat Softmax**	**Hierarchical**
Complexity	$O(\|V\|)$	$O(\log \|V\|)$
Parameters	W, W'	W, Θ_{nodes}
Best For	Small Vocab	Large Scale Training
Strategy	Global Norm	Path-based Product

Bridging the Gap: Implicit Factorization

- **SVD (Global):** Explicitly factorizes a Term-Document or Co-occurrence matrix.
- **SGNS (Local):** Iteratively updates vectors based on local context windows.

The Levy & Goldberg (2014) Discovery

Skip-gram with Negative Sampling is implicitly factorizing a **Shifted PMI Matrix**:

$$\mathbf{v}_w \cdot \mathbf{v}'_c \approx \text{PMI}(w, c) - \log k$$

- This proves that "neural" embeddings are grounded in the same distributional statistics as classical SVD.
- The dot product in the embedding space directly estimates word association strength.

GloVe: Global Vectors for Word Representation

- **Motivation:** SVD uses global statistics but poor at word analogies; Skip-gram is great at analogies but ignores global statistics.
- **Core Insight:** Meaning is found in the **ratios** of co-occurrence probabilities, not just raw counts.

The GloVe Objective Function

$$J = \sum_{i,j=1}^{V} f(X_{ij})(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

- X_{ij} are global co-occurrence counts.
- $f(X_{ij})$ is a weighting function to prevent common words (like "the") from dominating.
- It effectively performs a weighted least squares regression on the log-counts.